

ProofOps

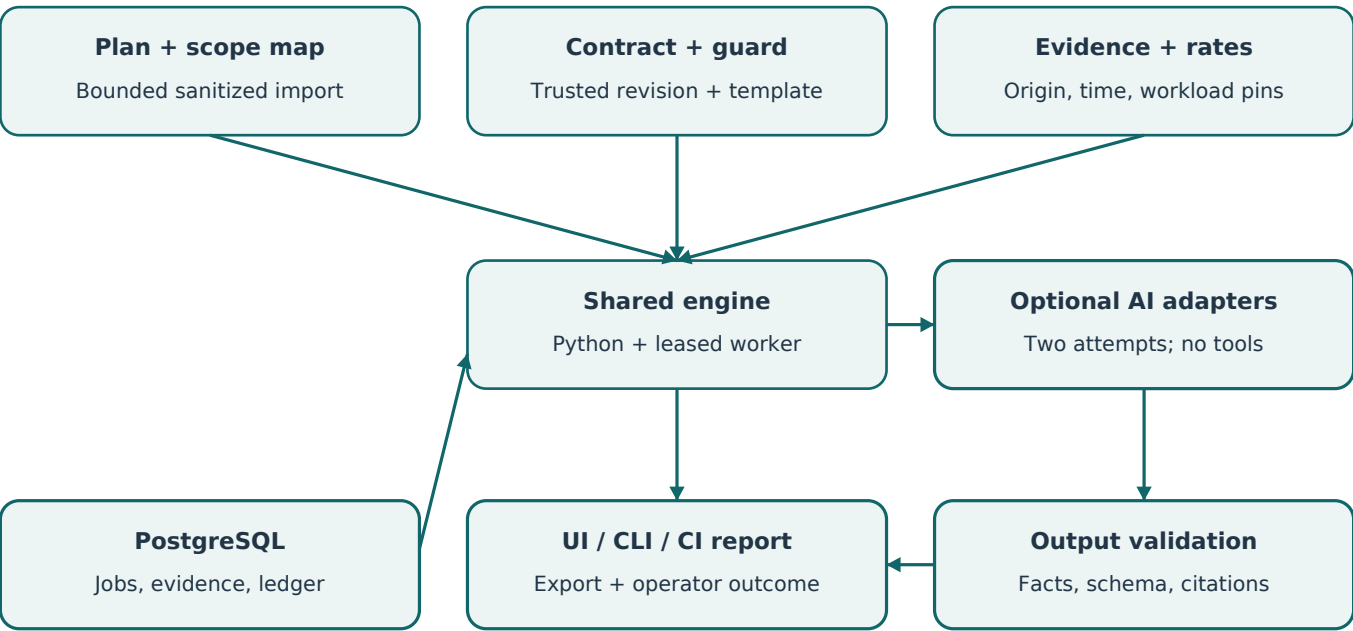
Technical and learning guide

Evidence for a specific infrastructure change

Working local software. Synthetic review fixtures. Measured local Docker experiments. Live AWS and paid model evaluation unrun.

Build date: 2026-10-05 · Application version 0.1.0

Editable Markdown, machine inventories and saved result extracts accompany this searchable guide. Source links and section links are clickable.



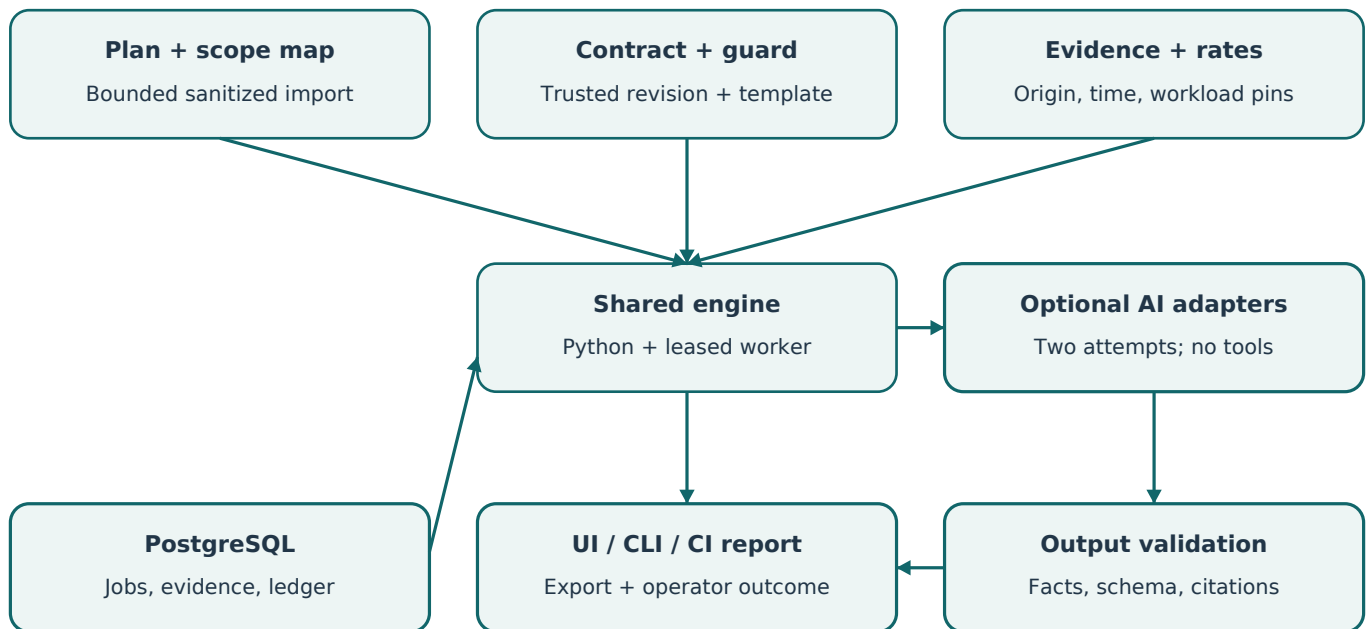
Human review > draft fixtures > separate protected source review. No app deployment.

Contents

Architecture and data flow	3
Languages, tools and concepts	4
Actual software inventory	6
Module reference	7
Model integration and inventory	11
AWS collection and optional isolated demo	13
Testing and measured evidence	14
Recorded results	18
Evaluation and economics	20
Learning path	22
Operations and troubleshooting	23
Owner and interview walkthrough	26
Data sources and boundaries	28

Architecture and data flow

ProofOps is a local, single-operator review application for one explicitly mapped Linux x86_64 ECS Fargate service. A change starts with a bounded input bundle. The shared Python engine owns normalization, Decimal costs, coverage, performance comparison and the final outcome. React reads persisted backend state.



Human review > draft fixtures > separate protected source review. No app deployment.

The API accepts JSON replay IDs or ZIP uploads, assigns IDs, and queues jobs with a scope-bound idempotency key. Reusing the same key/body returns the same job; a changed request returns 409. The worker claims with PostgreSQL FOR UPDATE SKIP LOCKED, records its lease, renews bounded work and writes the deterministic report separately from AI prose. A reclaimed dispatched model call becomes uncertain; its money remains reserved and the call is not sent again.

The only enforcing guard family is `ecs_task_memory_floor`. A trusted revision binds service scope, contract hash, image/profile/dependency applicability, approved bound and exact fixed Rego template hash. A UI draft and fixture pass never activate a guard. The API can export a draft for separate review. Existing reports become historical when their commit/policy/time basis changes.

Decisions and invariants

1. An applicable known constraint/performance violation wins: `revise_change`.
2. Otherwise missing, stale, incompatible or unknown required evidence yields `collect_evidence`.
3. Complete passing supported evidence yields `request_review`, displayed as Ready for engineering review.
4. Unsupported scope yields `out_of_scope`, a coverage result. Invalid inputs/internal errors are job failures.

Collection status, freshness and sufficiency are separate dimensions. A successful empty query is not zero errors. Evidence timestamps use UTC. Replay uses a recorded evaluation time; a current decision

uses current time. Origin (synthetic_fixture, benchmark, local_observation, aws_observation) is separate from execution mode.

Costs use Decimal rates, vCPU/MiB conversions and explicit billable task-hours. They cover task CPU/memory, with excluded charges shown. Changed container limits alone do not lower task allocations. Unknown usage/rates remain null. Monthly projected costs are never divided by an unrelated short-run request count.

Persistence and boundaries

PostgreSQL stores contracts, changes, normalized evidence, workload runs, leased jobs, immutable reports, guard revisions/drafts, outcomes, attempts, budget ledger/cache, billing rows and audit events. Alembic is required before readiness succeeds. Content-addressed ZIP artifacts live outside the web root in artifacts/; downloads resolve server-assigned hashes, not client filesystem paths.

The raw Terraform plan is parsed in memory and discarded after extracting a bounded allowlist. Sensitivity/unknown metadata is retained as paths; values stay unresolved. ZIP imports reject unknown filenames, duplicate paths, traversal, symlinks, encryption and expansion beyond limits. JSON rejects duplicate keys, non-finite numbers and excessive nesting. Nothing in an imported plan/repository is executed.

Model context is built from matched report facts and original development cards/examples. It contains no research evaluator paths, labels or revealing case IDs. Provider adapters have no tools. Mechanical schema/citation checks do not prove causal reasoning; evaluator annotations score semantics separately. API/worker containers do not contain the evaluation corpus.

The local controls do not implement multi-user identity, tenant isolation or public hosting. Real team enforcement requires authenticated roles, retention policies, deployment-specific authorization and protected trusted workflows. Kubernetes, other services/clouds, autonomous deployment and trained routing remain outside this build.

Languages, tools and concepts

A **model** is a remotely hosted learned function that generates an answer from input. An **SDK** is client code that sends requests to that service. A **framework** supplies application structure. A **database** persists/query-controls state. A **cloud service** supplies an external capability such as ECS compute or CloudWatch telemetry. Installing the OpenAI SDK does not configure a model, create access, or establish quality.

Languages and formats

Language / format	Actual files and purpose	Concepts to learn
Python	backend/, scripts/, evaluation/, tests/; domain, API, worker and tooling	Functions, modules, typing, exceptions, context managers, Decimal, UTC and tests
TypeScript / JavaScript	frontend/src/, tests/e2e/, testing/virtual_users.js; browser UI/automation and k6	Types, async requests, React state/effects, event handlers and independent response oracles
SQL	SQLAlchemy-generated PostgreSQL operations and Alembic migrations	Keys, joins, transactions, row locks, isolation, JSONB and indexes

Language / format	Actual files and purpose	Concepts to learn
HTML / CSS	frontend/index.html, React markup, frontend/src/styles.css	Semantic elements, labels, layout, focus, responsive sizing and contrast
Terraform HCL	infra/aws-demo/*.tf; optional dedicated AWS resources	Providers, resources, variables, plan/apply/state and ownership
Rego	policies/templates/ecs_task_memory_floor.rego; fixed memory rule	Declarative predicates, input data, deny results, scope and exceptions
JSON / JSONL	Typed inputs, rates, manifests, inventories, result exports	Schemas, strict parsing, canonical hashes, null versus zero, newline records
YAML	compose.yaml, GitHub workflow	Declarative services/jobs, quoting, environment interpolation and trust boundaries
Shell / PowerShell	README run commands and CI steps	Working directory, quoting, exit codes, environment variables and process lifetime

Why these dependencies exist

FastAPI exposes the typed HTTP API; Uvicorn runs it. Pydantic and JSON Schema reject invalid contracts; pydantic-settings reads environment configuration. React renders UI state; Vite builds assets and TypeScript checks browser interfaces. SQLAlchemy manages database access, psycopg transports PostgreSQL queries, and Alembic tracks schema migrations. PostgreSQL provides real concurrent transactions rather than pretending SQLite has identical locks.

boto3 implements AWS request signing and service APIs. Official OpenAI and Anthropic SDKs implement their different structured-output contracts behind one app interface. HTTPX supports bounded HTTP calls and SDK-boundary mocks. The domain logic remains independent of these provider clients.

Docker/Compose packages local services; uv and npm lock application dependencies. Terraform and its locked AWS provider describe optional isolated cloud scaffolding. GitHub Actions runs the defined offline pipeline. Conftest executes OPA/Rego policies; k6 offers controlled load and preserves latency/error/generator metrics. pytest runs Python tests, Hypothesis supplies meaningful generated input cases, and Playwright drives Chromium through real UI workflows. Ruff checks/formats Python, mypy checks domain types, TypeScript builds the web contract, and Prettier formats browser/workflow source.

ReportLab creates selectable vector/text PDFs; PyMuPDF verifies text, fonts, links and page geometry; Pillow composes rendered-page contact sheets for visual review. These are documentation-only dependencies in the docs group. None is required to serve a review.

Actual installed/locked versions and required/optional roles are generated into docs/software_inventory.json and the PDF tool table. uv.lock, frontend/package-lock.json, .terraform.lock.hcl and the tool download checksums are authoritative for the corresponding dependencies. Host Node was 25.7.0; the Docker frontend and recommended development setup use Node 22 LTS. The actual host is macOS ARM64; containers run Linux.

LangGraph, Kubernetes, vector databases, a trained router and general multi-cloud discovery are deferred. No code silently depends on them.

Actual software inventory

Tool	Version	Role
fastapi	0.142.2	Required API framework
uvicorn	0.54.0	Required ASGI HTTP server
pydantic	2.13.5	Required typed data contracts
pydantic-settings	2.15.0	Required environment settings
sqlalchemy	2.1.3	Required database mapping/transactions
alembic	1.20.0	Required database migrations
psycopg	3.3.6	Required PostgreSQL driver
boto3	1.43.108	Optional live AWS client; required adapter dependency
openai	2.54.0	Optional live provider; required SDK adapter dependency
anthropic	0.125.0	Optional live provider; required SDK adapter dependency
httpx	0.28.1	Required bounded HTTP client and SDK test transport
jsonschema	4.26.0	Required JSON Schema validation
pytest	9.1.1	Development test runner
hypothesis	6.168.3	Development generated-input tests
ruff	0.16.10	Development lint/format checks
mypy	1.20.2	Development Python type checks
reportlab	4.5.1	Documentation PDF generation only
pymupdf	1.28.2	Documentation PDF validation/rendering only
pillow	12.3.0	Documentation contact sheets only
react	19.3.0	Required browser UI framework
react-dom	19.3.0	Required DOM renderer
typescript	5.9.3	Development browser type checker
vite	7.3.6	Development asset build/server
@playwright/test	1.63.0	Development browser automation

Tool	Version	Role
prettier	3.6.2	Development source formatter
lucide-react	0.468.0	Required interface icons
conftest	0.71.0	Required deterministic guard fixtures
k6	2.3.0	Required measured workload profiles
terraform	1.16.5	Optional isolated AWS scaffold
Python	3.12.15	Required; application range 3.12.x
uv	0.12.23	Required frozen dependency installer
Node	22 (container/LTS setup); host 25.7.0	Frontend build; not a backend runtime
PostgreSQL	17.6-alpine	Required persistent transactional database
nginx	1.28-alpine	Container static web/proxy server
Terraform AWS provider	6.67.0	Optional isolated AWS scaffold; not deployed
Docker	Docker version 29.6.1, build 8900f1d	Required local container runtime; host version observed, not package-locked
Compose	5.2.0	Required local container runtime; host version observed, not package-locked

Module reference

Python module paths such as `domain/engine.py` are relative to `backend/proofops/`; fixture, frontend, script and documentation paths are relative to the app root. The API, CLI and CI share typed records and the same review function. Evaluation and experiment-runner scripts are explicitly invoked tools excluded from the runtime image; the separate workload fixture is packaged for the optional workload service. Test names below are executable references, not claims about live accounts.

Contracts and configuration

Files: `domain/schemas.py`, `domain/common.py`, `config.py`, `contracts/reports-demo.json`.

Inputs are versioned JSON, environment settings and explicit UTC reference times. Pydantic validates types, record sizes, Decimal amounts, request counts and resource identities. Canonical serialization produces hashes used by reports, cache keys and exports. Outputs are typed records with distinct missing, unknown and zero values. Configuration limits artifacts to the app's artifact directory and prices to `config/`. Invalid records produce field errors rather than a favorable review. Tests:

test_strict_allocation_roundtrip, test_unsupported_numbers_stay_unknown,
test_ambiguous_or_deep_json_is_validation_error.

Terraform normalization

File: normalization/terraform.py.

Inputs are terraform show -json bytes, an exact service map, repository and commit identities. The importer selects the mapped ECS task definition, preserves actions/unknown/sensitive paths, validates supported Linux x86_64 CPU/MiB combinations and hashes the non-resize configuration. It extracts only needed facts, then discards the raw plan. Outputs are a ChangeSet and explicit normalization findings. Unknown expressions, sensitivity, mixed images, changed application configuration and unsupported deployment shapes prevent a simple resize claim. Nothing in the imported repository is executed. Tests: test_sensitive_plan_bytes_never_survive_import, test_computed_arn_change_is_not_a_workload_change, test_container_only_change_has_no_task_compute_saving.

Input bundles and replay artifacts

Files: storage/bundles.py, storage/artifacts.py.

Directory/ZIP inputs contain only allowlisted JSON records. Size, filename, duplicate-entry, symlink, expansion, nesting and finite-number checks run before typed validation. Outputs are sanitized content-addressed bundles with schemas, hashes, findings, source versions and the recorded reference time. Downloads use server-assigned artifact IDs. Replay verifies the saved result without AI or refreshing its approval. Historical ZIPs without versions.json remain readable; newly exported ZIPs include it. Tests: test_archive_paths_never_extract, test_archive_symlink_duplicate_count_and_expansion_limits, test_cli_review_and_replay_share_outcomes.

AWS and replay collectors

File: collectors/aws.py.

The AWS input is a configured identity/scope and normal boto3 credentials. STS verifies the account before bounded ECS, CloudWatch and Logs calls. The collector checks task revision/platform/allocation, redacts bounded log examples and stops pending Logs Insights queries at the call cap. Output preserves collection state and observation provenance. Denied, empty, partial, failed and pending data never silently becomes zero. The replay adapter returns recorded observations without cloud access. The collector is a CLI path; it does not deploy or automatically repair evidence. Tests in test_aws_collectors.py use actual SDK Stubber contracts, including account mismatch and query cancellation. Live AWS remains unrun.

Deterministic review and coverage

File: domain/engine.py.

Inputs are a validated ReviewInput and separately trusted revision. The engine checks identity, applicability, freshness, required sources, cost, workload and memory policy. An applicable known violation wins over missing evidence; otherwise missing evidence, supported passing evidence and unsupported scope map to the other three outcomes. Outputs are immutable findings, coverage and a report core. A model cannot overwrite the outcome. Tests: test_three_report_paths, test_violation_precedes_missing, test_collection_states_do_not_become_zero, test_empty_stale_and_new_commit.

Dated task cost engine

File: domain/costs.py.

Inputs are matching region/architecture/OS/purchase-option rates and explicit baseline/candidate task-hours. Decimal arithmetic multiplies task-hours by vCPU and GiB rates; percentage is null for a zero baseline. Output identifies covered CPU/memory charges and excluded network, logs, storage, taxes and commitments. Different task-hours are recomputed; missing inputs stay null. Cost per correct request is emitted only when cost and request windows are comparable. Tests: `test_cost_null_zero_and_changed_hours`, `test_container_only_change_has_no_task_compute_saving` and frozen evaluation cost predicates.

Workload comparator

File: `domain/performance.py`.

Inputs are baseline/candidate `WorkloadRun` records and the frozen service contract. The comparator binds image, configuration, profile, dependency, platform, environment and warmup, then checks each run and request class separately. Outputs expose correct/completed/offered counts, latency, failures, restarts and sufficiency. It never averages p95 values. An undersampled run cannot establish capacity; a known failure cannot hide behind a reduced correct count. Tests: `test_smoke_does_not_establish_capacity`, `test_failures_cannot_hide_breach_by_reducing_correct_count`, `test_local_arm_measurements_cannot_validate_x86_contract`, `test_p95_equality_fails_and_no_average`.

Policy lifecycle and Conftest

Files: `policies/guards.py`, `policies/fixtures.py`, `policies/templates/ecs_task_memory_floor.rego`, `policies/approved/reports-demo.json`.

The one supported `ecs_task_memory_floor` specification binds exact service/workload scope, approved bound, incident/repair references, owner, reviewer, expiry and template hash. Drafting preserves those facts and leaves the state as draft. Fixture validation uses the fixed template and ten generated cases. Export checks the exact validated proposal, template and fixture hashes. Source-control review outside the app is required for activation. New commits/images/contracts or expired exceptions invalidate applicability. Tests: `test_actual_rego_fixture_suite`, `test_always_deny_fails_healthy_fixtures`, `test_trusted_policy_cannot_be_weakened_by_input`, `test_guard_draft_validation_export_and_tamper_rejection`.

Model context and explanation validation

Files: `models/context.py`, `models/explanations.py`, `models/development/`.

Inputs are already-computed report facts, bounded evidence summaries and curated development cards/examples. Compact and bounded-flat variants serialize the same evidence facts and record omissions. Outputs are prompts and a strict explanation contract. Validation checks exact evidence IDs, finding codes, numeric facts and next step; it rejects unknown fields. Retrieval never reads evaluator labels or fault-bearing benchmark paths. Quoted log text grants no authority. These mechanical checks do not establish causal truth. Tests: `test_model_cannot_change_facts_or_action`, `test_injected_log_instructions_get_no_authority_or_label_access`, `test_valid_citations_do_not_validate_causal_inference`.

Provider adapters and bounded router

Files: `models/adapters.py`, `models/router.py`, `config/model_prices.json`.

Adapters translate the shared request into OpenAI Responses or Anthropic Messages using official SDKs, closed schemas, explicit output limits and no tools. They normalize completion, refusal, truncation, quota, throttle, timeout and usage. The router first handles scope/missing-evidence/template cases, then permits at most two attempts with an overall task deadline. Permanent failures do not loop; transient retry uses the same two-attempt allowance. A routed mechanical failure can use the configured strong slot. Confidence never controls acceptance.

Both providers failing leaves the deterministic report and a visible AI-unavailable state. Tests in `test_provider_adapters.py` exercise SDK HTTP boundaries; `test_routing_bounded_attempts_cache_and_scope`, `test_transient_retry_consumes_the_second_attempt` and `test_both_provider_results_unusable_preserves_deterministic_guard` use PostgreSQL.

Budget ledger and accepted-output cache

File: `models/budget.py`; cache integration in `models/router.py`.

Inputs include positive overall/per-task limits, exact model/capability prices and a stable task identity. PostgreSQL transactions reserve conservative planned cost before dispatch and reconcile reported uncached/cache-read/cache-write/output usage afterward. Timeouts/crashes keep ambiguous amounts pending. Restarting does not reset spend. Actual cost exceeding a reservation is recorded, not concealed. Cache identity includes scope, inputs, policy, time basis, mode, prompt/schema, provider/model and price version; only complete validated outputs are cached for five minutes, retaining original usage. Tests: `test_atomic_budget_reservations_and_duplicate_dispatch`, `test_same_reservation_key_cannot_double_spend`, `test_nonretryable_and_uncertain_attempts`.

PostgreSQL repository and migrations

Files: `storage/database.py`, `storage/repository.py`, `migrations/`.

SQLAlchemy maps normalized records and bounded JSONB data to PostgreSQL. Alembic creates/version-controls tables for jobs, reports, revisions, artifacts, outcomes, attempts, budgets, cache, billing and audit. Idempotent requests bind body and scope; conflicting reuse fails. Connection/pool/statement/lock timeouts bound operations. Tests run on `proofops_test`, never the operator database. Tests: `test_api_worker_report_and_download`, `test_same_reservation_key_cannot_double_spend`, `test_job_claims_and_expired_lease_recovery`.

Persisted worker

File: `workers/runner.py`.

The worker claims a queued or expired job with `FOR UPDATE SKIP LOCKED`, renews its lease, runs bounded stages and persists the deterministic result before optional prose. A maximum claim count prevents endless recovery. Reclaiming a dispatched model attempt preserves uncertain charges and never blindly redispaches it. Outputs are persisted stage, report/artifact IDs and sanitized errors. Tests: `test_job_claims_and_expired_lease_recovery`, `test_worker_crash_after_dispatch_keeps_charge_pending_and_does_not_retry`.

API and command line

Files: `api/app.py`, `cli.py`, `domain/summaries.py`.

FastAPI handles imports, review creation/list/detail, bundle downloads, draft/validate/export, outcomes and billing analytics. Health is distinct from database readiness. Request byte caps, origin checks and typed IDs bound the local interface. The CLI exposes review, replay, guards test, collect, ingest-costs and explicit evaluate; summaries share the engine's outcomes and exit codes. Evaluation launches a separate program rather than adding labels to runtime imports. Tests: `test_origins_paths_validation_and_unknown_ids`, `test_cli_evaluator_paths_rejected_without_reading`, `test_cli_trusted_map_cannot_be_changed_by_input`.

UI and analytics

Files: `frontend/src/main.tsx`, `api.ts`, `types.ts`, `styles.css`; `storage/analytics.py`.

React renders persisted reviews, evidence detail and outcomes. Users can compare costs/coverage/classes, inspect citations and attempt accounting, export a validated draft, detect changed-commit applicability and record a disposition. Stale responses cannot replace newly selected detail. Save success is shown only after the API succeeds; unavailable analytics stays unavailable. FOCUS ingestion preserves Decimal values, negative credits, nulls, currencies and row position; repeated file imports are idempotent. The sample is never attributed to the demo service's invoice. Tests: browser scenarios in `tests/e2e/reviews.spec.ts`, `test_focus_ingestion_is_idempotent_and_reconciles`, `test_billing_endpoint_and_mixed_currency_null_duplicate_rows`.

Workload fixture and experiment runner

Files: `demo/reporting_api/server.py`, `testing/virtual_users.js`, `scripts/run_workload.py`.

The separate HTTP service sums bounded integer-cent items. k6 supplies an independent oracle and an open arrival-rate schedule with small/medium/large classes. The runner starts only labelled, cgroup-limited local containers, performs warmup, captures Docker state, freezes calibration and alternates three comparison pairs. A deliberately oversized startup allocation tests actual OOM and repair; pressure is opt-in. It preserves native summaries, logs, hashes and terminal states. These local Linux ARM64 measurements cannot validate the AWS x86_64 contract. See the recorded workload table and commands in `docs/testing.md`.

Evaluation, CI, infrastructure and documentation

Files: `evaluation/`, `.github/workflows/offline.yml`, `scripts/ci_review.py`, `infra/aws-demo/`, `scripts/build_docs.py`, `scripts/validate_docs.py`.

The frozen 60-case evaluator keeps labels outside prompts, scores deterministic/explanation/guard tasks separately and marks unavailable provider work unrun. Rescoring saved outputs binds independent annotations to input and output without another call. CI runs offline tests and a recurrence example using base-revision code/policy/map against bounded candidate data; production enforcement still needs protected workflow settings. Optional Terraform validates dedicated, zero-task AWS scaffolding but was not deployed. Documentation builds embedded-font searchable PDFs from Markdown, inventories and saved results; validation renders every page and checks text, fonts and links. Tests: `test_frozen_split_has_no_group_leakage`, `test_replay_rescore_keeps_outputs_and_never_calls_a_provider`, local CI regression, Terraform validate, and the PDF validation artifact.

Model integration and inventory

No exact live model ID is configured or evaluated for this handoff. `docs/model_inventory.json` records empty model slots, source dates, reasons for unknown prices/measurements and concrete intended tasks. OpenAI is the default provider slot for a first attempt and Anthropic for a bounded escalation; both roles can be reassigned. This is configurable routing plumbing, not a finding that either brand is cheaper, stronger or better.

How hosted inference is used

The app serializes a system instruction, bounded deterministic facts and a JSON Schema, then sends them to a provider endpoint. A hosted model generates output tokens; the SDK returns content, completion status and usage. The application sees the request/response contract, not proprietary parameter counts, training data, internal architecture or private reasoning. No assumptions about those hidden details are needed.

For explanation, input includes the exact outcome, allowed numeric facts, finding codes, evidence references, coverage and curated development material. The result is a typed explanation with citations and a next step. For guard drafting, input is the already-approved scope/bound/reference specification; output must preserve it exactly in draft state. Example: explain why a 1,024 MiB candidate violates a supplied 2,048 MiB floor, or re-express that exact approved floor as a draft. Neither task discovers a safe bound or executes a change.

OpenAI uses the Responses API with `text.format`, a named strict schema, `store=false`, a maximum output and no tools. Anthropic uses Messages with separate system/user input, `output_config.format`, a maximum output and no tools. Complete non-streaming output is required. Provider grammar constraints are followed by application schema, citation, numeric and action checks. Refusal, truncation and unsupported/permanent errors stay explicit. No model output can replace a deterministic result, activate a guard, read evaluator files or run commands.

Routing, costs and limitations

Template behavior is the default. Out-of-scope or missing required evidence stops unnecessary explanation calls. An eligible configured language task may use cheap-only, strong-only or routed policy. Routed mode starts with the configured first slot, and can escalate after mechanical failure; a transient retry consumes the same two-attempt maximum. Permanent quota/authentication/request errors and refusal do not create loops. Confidence and unsupported causal statements can still be wrong even when mechanics pass; live semantics require independent annotation.

Before dispatch, the app requires an exact allowlisted model, key from the environment, structured-output capability, official-source USD rates no older than 31 days, and positive overall/per-task budgets. A conservative byte/framing estimate reserves planned cost atomically. Actual reported input, cache-read, cache-write and output usage determines reconciled cost. OpenAI reasoning usage, where reported, remains included in output rather than charged twice. No explicit reasoning or hosted-tool capability is enabled. Provider billing differences can exceed a reservation; the ledger records this and retains uncertainty instead of claiming a guaranteed external invoice cap.

Cache hits preserve original usage and record current incremental cost separately. Failed/truncated output is never an accepted cache entry. Timeout/crash uncertainty retains reserved money. When both providers fail, the deterministic report and template next step remain available and AI unavailability is visible. Live quality, latency, parity and task-cost comparisons are null, not invented zeroes. Mocked official SDK requests have passed; they are not a model benchmark.

Sources checked 2026-10-05: [OpenAI structured outputs](#), [OpenAI pricing reference](#), [Anthropic structured outputs](#), [Anthropic pricing reference](#). No particular price or account-specific model access is asserted from these links.

Configured model slots

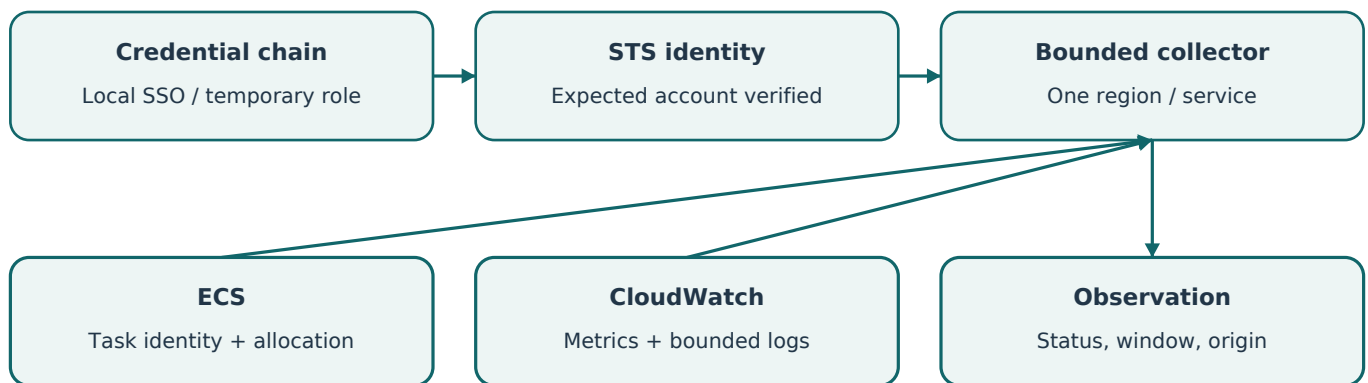
Slot / provider	Exact model and state	Measurements / prices
cheap / openai	ID: null; unconfigured	All live cost, latency, quality and model prices: null. No exact model configured and no live calls made; reference URLs do not supply a chosen price or measurement.
strong / anthropic	ID: null; unconfigured	All live cost, latency, quality and model prices: null. No exact model configured and no live calls made; no proprietary architecture, pricing or quality has been inferred.

No configured_models entries exist. SDK tests are mocked; these slots are not evaluated model choices.

AWS collection and optional isolated demo

The implemented collector calls STS GetCallerIdentity, ECS DescribeServices, DescribeTaskDefinition, ListTasks and DescribeTasks, CloudWatch GetMetricData, and CloudWatch Logs StartQuery, GetQueryResults and StopQuery. boto3 uses the normal temporary-credential/SSO/assumed-role chain. There are no deployment calls in the collector.

The account, region, cluster, service, task definition ARN, allocation, image and platform must match. Mixed task revisions, incomplete pagination and unknown OS/architecture remain insufficient. Service-average CPU/memory is context, not a correctness/latency measurement. Logs are bounded and redacted; empty, pending, denied and failed states remain distinct. A content hash proves identity, not source truthfulness.



Read-only adapter code tested with mocks; no AWS deployment or live smoke performed.

IAM and cost scope

[readonly-collector-policy.json](#) lists exactly the implemented action set using synthetic account/scope placeholders. Replace every identity with the separately reviewed intended scope. DescribeServices and starting a log query are resource scoped. The sample grants read-only task enumeration/description and metric/query lifecycle actions with wildcard resources and a region condition. Some metric/query operations require wildcard resources; the task-read permissions may be narrowed further using the relevant AWS action's supported resource types and the actual task family/cluster. This sample is functional action minimization, not a claim of perfect per-resource IAM isolation.

STS caller identity normally requires no identity-policy permission; the statement documents the call. Query result retrieval/stop uses query IDs, which the API resolves. Application checks also reject a different account/region/service. Do not substitute an administrator policy.

CloudWatch Logs Insights charges depend on bytes scanned, not just returned row count. CloudWatch API calls, logs, ECR storage, task compute, NAT/egress, endpoints, load balancers and retained infrastructure can all have separate costs. No current live AWS rate was fetched or cost measured. Budget alarms monitor; they are not a hard stop.

Terraform demonstration

`infra/aws-demo/` is optional dedicated demo HCL. It uses Terraform 1.16.5 and a locked AWS provider, an isolated name prefix, explicit expected account, private subnets/security groups supplied by the operator, Linux x86_64, a pinned ECR image, one-day log retention and **zero running tasks by default**. The task role has no application AWS permissions. The execution role can pull the declared repository and write the declared log group.

Local validation (no plan/apply):

```
.venv/bin/python scripts/install_terraform.py
.tools/bin/terraform -chdir=infra/aws-demo init -backend=false -input=false
.tools/bin/terraform -chdir=infra/aws-demo fmt -check
.tools/bin/terraform -chdir=infra/aws-demo validate
```

No AWS environment was deployed. A future authorized run needs a dedicated account/region, a built and verified Linux x86_64 image, exact private networking/egress, a short duration and an explicit spending allowance. Build a matching trusted contract, service mapping and collector configuration; the current synthetic guard is not an AWS approval. After reviewing a plan, a human can choose whether to apply and start one task. No deployment step is part of local setup.

Cleanup for a future deployment must use that exact directory/state: review `terraform -chdir=infra/aws-demo plan -destroy`, then explicitly authorize `terraform -chdir=infra/aws-demo destroy`. Existing subnets/security groups are inputs and are not owned by this module. Inventory ECR/networking/retained artifacts separately. Never destroy unrelated state or assume scaling to zero removes every charge.

Sources: [Fargate task sizing](#), [Fargate pricing](#), [ECS IAM actions](#), [CloudWatch IAM](#), [Logs IAM](#).

Testing and measured evidence

A test compares observed behavior with an independently justified expected result, or oracle. Passing a test supports only its stated scope. The saved verification and workload tables in `docs/results/results.md` are generated from actual artifacts. The full machine record is `docs/results/verification.json`. Native logs and run manifests remain under `artifacts/`.

Types of verification

Unit tests isolate decision/arithmetic/boundary behavior; property tests exercise many valid or invalid allocations. Integration tests use a real PostgreSQL database for transactions, idempotency, leases and concurrency. Policy tests execute actual Conftest/Rego, including healthy controls that make an always-deny implementation fail. SDK contract tests call mocked provider/AWS HTTP boundaries and verify parameters, completion states and accounting. They do not establish account access or model quality.

Browser tests exercise the real local UI/API/worker. Load tests measure correct work and latency under offered demand. Fault injection proves a specific controlled failure and repair. Security/resilience tests attempt traversal, malformed JSON, secret retention, prompt injection, cross-scope cache reuse and ambiguous worker recovery. AI evaluation scores independent labels and semantics separately from schema. Human usability requires an uncoached consenting engineer; it was not replaced by browser automation.

Commands and prerequisites

From the app root, install the frozen development dependencies and pinned Conftest/k6 as in the README. Start Docker PostgreSQL and create the dedicated test database with `python scripts/create_test_database.py`. Integration fixtures run Alembic migrations and refuse any database name other than `proofops_test`.

```
.venv/bin/pytest tests/unit tests/policy tests/integration -q --junitxml=artifacts/checks/backend.xml
.venv/bin/ruff check backend scripts tests evaluation demo
.venv/bin/ruff format --check backend scripts tests evaluation demo
.venv/bin/mypy backend/proofops
.venv/bin/proofops guards test
.venv/bin/proofops evaluate --manifest evaluation/manifest.json --mode replay --output artifacts/evaluation
```

Browser checks require API, worker and web at ports 8000/5173, plus Chromium:

```
cd frontend
npm ci
npm run build
npx playwright install chromium
npm run test:e2e
```

PowerShell uses `.venv\Scripts\pytest.exe`, `ruff.exe`, `mypy.exe`, `proofops.exe` and `python.exe` in place of `.venv/bin/...`. These exact PowerShell setup commands are documented but were not run on this macOS host. Backend JUnit and Playwright JSON are saved under `artifacts/checks/`; retained traces/screenshots are under `artifacts/playwright/` and `artifacts/screenshots/`. The Starlette TestClient emits one upstream httpx compatibility warning; no test is skipped because of it.

T01-T24 acceptance matrix

For a focused Python row, run `.venv/bin/pytest <path>::<test_name> -q`. All listed Python tests also belong to the full backend command above and its JUnit artifact. The common input root is `fixtures/replays/`; policy cases derive from the trusted fixture specification, and negative semantic labels live only under `evaluation/evaluator_only/`.

ID	Executable evidence / fixture	Result and scope
T01	<code>scripts/run_workload.py</code> all; <code>testing/virtual_users.js</code> ; <code>artifacts/workload/comparison.json</code>	Measured local normal/peak traffic; per-run and per-class results retained. This is not AWS evidence.
T02	<code>tests/unit/test_review.py::test_three_report_paths</code> ; <code>valid-resize/</code> ; <code>API/browser</code> equivalents	Valid synthetic candidate requests engineering review only after its contract passes. Local candidate experiment reported separately.
T03	<code>tests/unit/test_review.py::test_violation_precedes_missing</code> ; <code>unsafe-resize/</code> ; policy fixtures	1,024 MiB below the synthetic approved 2,048 MiB floor always revises; cost/AI cannot override.
T04	<code>scripts/run_workload.py</code> pressure; <code>artifacts/workload/pressure-repair.json</code>	Actual Docker-confirmed OOM under a 64 MiB limit with 128 MiB pressure, followed by verified repair at 256 MiB.
T05	<code>tests/unit/test_performance_boundaries.py::test_smoke_does_not_establish_capacity</code> ; <code>incomplete-evidence/</code>	A 20-request smoke run collects evidence under a 10,000-request contract.
T06	<code>tests/unit/test_review.py::test_collection_states_do_not_become_zero</code> ; <code>test_empty_stale_and_new_commit</code> ; <code>tests/unit/test_aws_collectors.py</code>	Denied/pending/empty/stale states preserved; mocked SDK collection plus deterministic checks. Live CloudWatch unrun.

ID	Executable evidence / fixture	Result and scope
T07	tests/unit/test_aws_collectors.py::test_sensitive_plan_by_tes_never_survive_import; tests/unit/test_review.py::test_unsupported_numbers_stay_unknown	Unknown/sensitive values remain unresolved and secrets do not survive import.
T08	tests/unit/test_review.py::test_container_only_change_has_no_task_compute_saving	Container-limit-only edits do not reduce task CPU/memory charges.
T09	tests/unit/test_review.py::test_cost_null_zero_and_changed_hours; frozen cost scenarios	Missing hours stay null; supported changed hours are recalculated.
T10	tests/integration/test_api.py::test_focus_ingestion_is_idempotent_and_reconciles; test_billing_endpoint_and_mixed_currency_null_duplicate_rows; fixtures/billing/focus_sample.csv	Real PostgreSQL ingestion verifies credits, nulls, currencies, identical rows and repeated imports.
T11	tests/unit/test_review.py::test_low_cpu_without_demand_evidence_does_not_justify_resize	Unit regression requires relevant demand/latency evidence; no live dependency-stall experiment claimed.
T12	tests/policy/test_memory_guard.py::test_actual_rego_fixture_suite; unrelated-service case	Exact named-service policy leaves another service alone; scope coverage stays explicit.
T13	tests/unit/test_review.py::test_trusted_policy_cannot_be_weakened_by_input; tests/unit/test_cli.py::test_cli_trusted_map_cannot_be_changed_by_input; scripts/ci_review.py	Trusted revision/map used despite candidate self-weakening. Local CI regression passes; remote enforcement unconfigured.
T14	tests/unit/test_review.py::test_empty_stale_and_new_commit; guard fixture cases; browser changed-commit test	Commit/image/config/expiry changes invalidate applicability while old replay remains reproducible.
T15	tests/unit/test_review.py::test_model_cannot_change_facts_or_action	Invented citation or numeric/action mutation is mechanically rejected.
T16	tests/unit/test_evaluation.py::test_valid_citations_do_not_validate_causal_inference	Authored unsupported-causality control passes mechanics but fails semantic annotation.
T17	tests/unit/test_provider_adapters.py::test_quota_and_throttle_are_distinct_and_sdk_does_not_retry; tests/integration/test_budget_jobs.py::test_transient_retry_consumes_the_second_attempt	Mocked permanent failures do not loop; transient retry consumes the bounded attempt allowance.
T18	tests/unit/test_provider_adapters.py::test_openai_terminal_states; test_anthropic_actual_sdk_mapping_and_terminal_states	Refusal/truncation/nonterminal output is not a successful cached explanation.
T19	tests/integration/test_budget_jobs.py::test_atomic_budget_reservations_and_duplicate_dispatch; test_worker_crash_after_dispatch_keeps_charge_pending_and_does_not_retry	Real PostgreSQL concurrency/recovery; uncertain spend remains reserved and undispached twice.
T20	tests/unit/test_import_security.py::test_injected_log_instructions_get_no_authority_or_label_access	Injected approval/deletion/disclosure instructions gain no tools or authority; runtime label boundary holds.
T21	tests/unit/test_evaluation.py::test_confident_wrong_answer_is_not_an_acceptance_signal	Authored confidently wrong control remains a semantic failure. No live cheap-model quality claim.

ID	Executable evidence / fixture	Result and scope
T22	tests/unit/test_review.py::test_mismatched_population_inconclusive; tests/unit/test_performance_boundaries.py::test_local_arm_measurements_cannot_validate_x86_contract	Incompatible units/timing/population/platform cannot establish improvement.
T23	tests/integration/test_budget_jobs.py::test_both_provider_results_unusable_preserves_deterministic_guard	Mocked failure of both providers leaves deterministic findings and visible AI unavailability.
T24	Manual uncoached workflow from docs/interview_walkthrough.md	Unrun: needs consenting engineer. Record completion, elapsed time, errors and help; no fabricated feedback.

Workload method and repeatability

The full k6 profile lasts eight minutes with an 80/15/5 small/medium/large mix. An open arrival-rate schedule offers 5–100 requests/second, allocating virtual users up to a cap. A virtual user is an execution slot, not an independent human. Offered rate and service latency jointly determine required concurrency. Dropped iterations reveal when the generator cannot begin scheduled work; slowing a closed loop would conceal this demand.

Warmup sends 100 verified requests before the measured window. The HTTP body carries a request ID, currency and bounded integer-cent items. The independent k6 oracle checks response ID/currency/count/total; a wrong HTTP 200 is an incorrect success, never useful work. p95 is a percentile of sampled latencies, not an average. Small/medium/large p95 and each repetition must meet the same limit; percentiles are not averaged across runs.

The frozen full contract requires at least 10,000 correct responses per run, p95 strictly below 250 ms, HTTP failures strictly below 1%, no incorrect successes, no dropped iterations and no restarts. Equality fails the two strict limits. Calibration uses a baseline before any candidate result. Three alternating baseline/candidate pairs use the same local image, profile and dependency hashes; baseline is 2 CPUs / 512 MiB and candidate 1 CPU / 256 MiB. These allocations are local cgroup settings, not Fargate task sizes or a cloud cost estimate.

```
docker compose --profile workload up -d --build workload
.venv/bin/python scripts/run_workload.py all
```

The first series is preserved under artifacts/workload/. To run a new independent series without overwriting it:

```
.venv/bin/python scripts/run_workload.py all --series repeat-01
```

To resume missing comparison runs after calibration, use `compare` with the same optional `--series`; frozen inputs must still match. Do not rebuild/change the workload image, server, profile or dependency lock mid-series. Full all takes about 57 minutes. Pressure uses only a newly owned labelled local container, never the API, database or AWS. A failed candidate remains failed; a missing summary is an execution failure, not a zero-latency pass.

Interpretation and unrun checks

Passing means the stated checks passed for the exact supplied inputs. A known breach is a failure. Insufficient/incompatible measurements are inconclusive, even if the code correctly chooses `collect_evidence`. Unrun means prerequisites were absent. Local Linux ARM64 Docker measurements cannot validate the synthetic Linux x86_64 AWS approval. No cloud invoice or realized savings was measured.

The FOCUS file is a separate 1,000-row sample: 13 negative billed rows, BilledCost 20.52022672899 USD and EffectiveCost 14.97651418586 USD. Tests derive those totals from the actual pinned input. They are not production dashboard constants.

Live AWS smoke, paid OpenAI/Anthropic quality/latency/cost, a downstream compact-versus-flat model comparison, remote GitHub Actions, Windows execution and human usability remain unrun. Their next prerequisites are documented configuration/account access and budgets, an authorized repository workflow, a Windows host, or a consenting peer as applicable. None blocks the supported local replay workflow.

Recorded results

Generated from actual saved artifacts on 2026-10-05T01:42:22.103526+00:00. This table describes the supported local workflow; live/unrun work is listed separately. Full JSON and native workload summaries accompany it in docs/results/.

Local verification

Check	Observed result
Backend, policy and PostgreSQL integration	94/94 passed; 0 failures; 0 skipped
Chromium browser workflows	6 passed; 0 unexpected; 0 skipped
Fresh local checkout/runtime environment	Passed; frozen runtime install and keyless review/replay
Deterministic synthetic evaluation	60/60 cases in 20 scenario groups
Terraform local validation	valid=True; 0 errors; 0 warnings; no plan/apply
Formatting/type/build/CI syntax	passed; exact commands and exits in verification.json
Research preservation	384 files checked; 0 changed/missing

Template and provider evaluation

These are synthetic descriptive results, not independent real incidents or model quality parity. The deterministic outcome's safety counts are separate from semantic prose scoring.

Task / policy	Observed count or status
Structured template explanations	57/57 mechanically valid; 3 additional out-of-scope abstentions
Narrow authored explanation/coverage rubric	60/60 correct; no live model prose scored
Applicable guard drafts	54/54 correct
Inapplicable guard cases	6/6 correctly returned no draft
Structured accepted-output coverage	111/120 template tasks

Task / policy	Observed count or status
Unsafe decision false negatives	0/18 unsafe cases
Healthy decision false positives	0/9 healthy cases
Correct evidence abstentions	30/30 insufficient cases
Template incremental API spend	USD 0; zero provider calls
Template local processing latency	p50 0.235 ms; p95 0.284 ms; sequential runner, no API queue
cheap_only	120 scheduled tasks unrun; live quality/cost/latency null
strong_only	120 scheduled tasks unrun; live quality/cost/latency null
routed	120 scheduled tasks unrun; live quality/cost/latency null

Measured local workload

Origin: local_observation, Linux ARM64 Docker on a shared development host. Baseline 2 CPUs / 512 MiB; candidate 1 CPU / 256 MiB. Same pinned local image/profile/dependencies, 100-request warmup, eight-minute full runs. No cloud cost or invoice measured. Each run/class is assessed separately; p95 values are not averaged.

Run	Correct responses	p95 (ms)	HTTP failures	Contract result
baseline-smoke-01	20	3.100	0	Smoke only; no capacity claim
baseline-calibration-01	20,399	5.942	1	Passed
baseline-full-01	20,399	5.980	1	Passed
candidate-full-01	20,399	5.481	1	Passed
baseline-full-02	20,400	5.934	0	Passed
candidate-full-02	20,399	6.166	0	Passed
baseline-full-03	20,399	6.262	1	Passed
candidate-full-03	20,399	6.190	1	Passed

Per-class p95 is checked independently against the same frozen 250 ms limit:

Comparison run	Small p95 (ms)	Medium p95 (ms)	Large p95 (ms)
baseline-full-01	5.265	6.452	10.828
candidate-full-01	4.955	5.868	9.299
baseline-full-02	5.193	6.401	10.680
candidate-full-02	5.397	6.701	10.966
baseline-full-03	5.472	6.752	11.160

Comparison run	Small p95 (ms)	Medium p95 (ms)	Large p95 (ms)
candidate-full-03	5.394	6.710	11.127

Comparison complete: True; all six comparison runs passed: True. Actual bounded OOM plus repair confirmed: True. Inspect native summaries for every request class, failure count and dropped iteration. Frozen thresholds: at least 10,000 correct; p95 below 250 ms; HTTP failures below 1%; zero incorrect successes, dropped iterations or restarts.

The OOM experiment injected a 128 MiB startup allocation into a 64 MiB cgroup and then verified 20 correct responses at 256 MiB. This controlled failure does not establish a 2,048 MiB AWS memory bound. The local resize is a resource-allocation comparison, not measured cloud savings.

Unrun prerequisites

Live AWS needs an exact authorized service/account and spending allowance. Paid model experiments need verified exact model IDs, credentials, current prices, positive budgets and independent semantic review. Remote CI needs a repository and protected workflow configuration. Windows commands need a Windows host. T24 needs a consenting uncoached peer; no feedback was fabricated.

Evaluation and economics

The frozen corpus is manifest.json: 60 original synthetic cases from 20 scenario groups, three variants each. Eight groups are development, four routing calibration and eight held-out. A group never crosses splits. evaluator_only/labels.json contains independently authored outcomes, finding predicates, guard specifications and rationales. The generator does not call the engine to produce expected labels. One documented vocabulary correction occurred before the first run; the manifest records it. Do not regenerate or tune against held-out results and keep calling them untouched.

The current corpus checks one guard family and variations on explicit constraints, not general incident diagnosis. The original research's eight worked examples remain development material. No RCAEval index, evaluator label, revealing file path or case ID is submitted to a model. API and worker images do not include this directory. Only explicit evaluation/test commands may read labels.

Reproduce the local run

```
.venv/bin/proofops evaluate --manifest evaluation/manifest.json --mode replay --output artifacts/evaluation
```

This writes summary.json, deterministic.json, contexts.json and results.jsonl. The three provider policies are marked **unrun** when replay is requested or live prerequisites are absent. Null cost/latency/quality means unmeasured; it does not mean free, fast or correct. template runs without keys or PostgreSQL. The standard CLI returns failure if any deterministic case disagrees with its oracle.

Each case schedules an explanation task and a guard/applicability task for template, cheap-only, strong-only and routed policies. Records include group/split, immutable input hash, expected label, actual output, prompt/schema/model/price metadata, route, provider attempts, usage, latency and known/pending cost. Provider failure and deterministic fallback remain visible in the record. Split-specific summaries preserve held-out denominators.

The local template run has 60/60 deterministic outcomes/predicates, 57 structured explanations plus three correct out-of-scope abstentions, and 54 applicable draft specifications plus six correctly inapplicable guard cases. It therefore has 111 structured accepted outputs across 120 scheduled template tasks. The other 360 scheduled tasks are unrun. These counts are regenerated from saved results in the PDF tables, not used as dashboard constants.

What each score means

Deterministic correctness checks expected outcome, required findings and specified cost predicates. Unsafe false negatives, healthy false positives and evidence abstentions in the summary refer to the engine's decision; they do not measure the safety of generated prose. Mechanical validity checks schema, exact references and numeric facts for structured explanations. Semantic correctness asks whether the explanation's meaning is supported. A valid citation can accompany an unsupported causal claim; the negative-control tests deliberately prove that distinction.

Known deterministic templates are scored against a narrow authored rubric. New model prose without an independent annotation receives `correct: null`. A stronger model's agreement or self-confidence never creates a truth label. Guard scoring uses the exact independently supplied scope, memory floor, references and draft state; inapplicable cases are counted separately from actual drafts.

Cost per attempted task includes failed calls, retries and escalation. Cost per correct explanation divides the total explanation-task cost, including failures, by independently correct explanations. Any unresolved charge keeps totals/cost-per-correct null. A second attempt is not automatically an escalation: changing provider/model is reported separately from a same-model retry. Local evaluator latency includes engine, context/retrieval and routing/SDK work, but has no API queue; persisted UI jobs expose queue timestamps separately.

Annotate saved outputs without paid reruns

Create an annotation JSON inside application artifacts/. Copy the exact `annotation_key` from a saved explanation record; it binds the full output, task and input hash. A reviewer must inspect the relevant evidence and write a Boolean judgment plus a rationale. Do not copy this structural example as a real judgment:

```
{
  "<annotation_key from the saved record>": {
    "correct": false,
    "reviewer": "<reviewer identifier>",
    "rationale": "<specific claim and evidence supporting the judgment>"
  }
}

.venv/bin/python -m evaluation.rescore --results-dir artifacts/evaluation --annotations artifacts/annotations.json
--output artifacts/evaluation-rescored
```

The destination must be new. The command preserves original outputs, usage, costs and timing; it makes no provider call. A judgment cannot leak onto the same prose with different evidence. Tests include deliberately wrong annotations only as scorer controls; they are not claimed as peer review. Live evaluation uses `python -m evaluation.run --mode live` only after IDs, keys, dated prices and explicit positive budgets are configured. No live provider experiment was run for this handoff.

Context experiment and uncertainty

The runner prepares compact and bounded-flat contexts from the same underlying report evidence. It records byte counts, hashes and omissions. If a representation exceeds the cap, it is excluded, not scored as model failure. Serialization bytes and the research's reference-token counts are not billed API tokens. The current experiment establishes reproducible inputs only; downstream semantic quality and token-cost comparison remain unrun.

Three variants per group are correlated. Descriptive 60-case accuracy cannot imply 60 independent incident trials or broad quality parity. Even zero errors in 30 independent accepted cases has a one-sided 95% upper error bound near 9.5%; 300 such cases lowers it to about 1%. Those are illustrative statistical bounds, not confidence intervals for this synthetic corpus. A future study needs

additional independent scenarios, frozen development/calibration decisions, independent reviewers and sufficient accepted held-out coverage, including cheap-model failures.

Sources: [RouteLLM](#), [FrugalGPT](#), [OpenAI structured outputs](#), [Anthropic structured outputs](#).

Learning path

The build order assumes an already comfortable Python/Docker developer. A beginner should use the stages below at their own pace; seven build days are not seven days to master the stack. Start with the deterministic replay, then learn the optional live boundaries.

Stage	Learn and why	Exercise and completion check
1. Files, terminal and Git	Paths, processes, environment variables, diffs and small commits explain where state lives.	Run <code>git status</code> inside the app, locate the three replays, and explain why a populated <code>.env</code> is ignored. Do not edit research.
2. Python and types	Functions, exceptions, modules, dictionaries and Pydantic turn raw JSON into explicit contracts.	In a new scratch test under the app, construct an invalid CPU/MiB pair. Confirm validation rejects it without coercing it to zero.
3. HTTP and structured data	Requests, status codes, JSON, schemas, UTC and Decimal distinguish transport success from business correctness.	Send a valid integer-cent report request, then an invalid body. Explain why an HTTP 200 with a wrong total fails the workload oracle.
4. Domain reasoning	Pure functions, invariants and precedence make decisions reproducible without an LLM.	Run all three replay commands. Explain why a missing source cannot cancel a known memory violation and why <code>request_review</code> is not deployment approval.
5. SQL and transactions	Tables, foreign keys, JSONB, locks and isolation protect jobs and money across processes.	Run the reservation/concurrency tests on <code>proofops_test</code> . Explain why two workers cannot spend the same reservation.
6. Web UI	TypeScript, React state, effects, accessibility and CSS connect visible state to persisted backend results.	Inspect evidence and record a disposition. Simulate an unavailable API and confirm the page does not display an invented zero or saved state.
7. Infrastructure as code	HCL, plan JSON, Fargate task sizes, IAM and container digests bind a proposed change to a resource.	Inspect the unsafe plan and service map, then run <code>Conftest</code> fixtures. Explain why another service is not rejected by this service's memory guard.
8. Performance experiments	Arrival rate, virtual users, p95, warmup, sample size and compatible populations define what a test can show.	Compare one baseline and candidate manifest. Identify offered/completed/correct/dropped work and explain why local ARM64 is not an AWS x86_64 bound.

Stage	Learn and why	Exercise and completion check
9. Optional LLM inference	Hosted model requests, structured output, tokens, pricing, mechanical versus semantic validation and fallback.	Run the mocked adapter and false-causality tests. Describe a response that passes schema/citations but is still wrong. No paid key is needed.
10. Evaluation and operations	Group splits, independent labels, accounting uncertainty, CI trust, logs, migrations and retention.	Reproduce offline evaluation, inspect the 8/4/8 groups and null live metrics, rebuild PDFs and recover a test worker lease. Explain one remaining production prerequisite.

Official starting points

- [Python tutorial](#) and [venv](#)
- [Git book](#) and [PowerShell introduction](#)
- [HTTP overview](#), [JSON](#), [TypeScript handbook](#)
- [Pydantic concepts](#), [FastAPI tutorial](#), [SQLAlchemy tutorial](#)
- [PostgreSQL tutorial](#) and [Alembic tutorial](#)
- [React learn](#), [Vite guide](#), [MDN CSS](#)
- [Docker getting started](#), [Terraform tutorials](#), [ECS guide](#)
- [Conftest](#), [Rego introduction](#), [k6 testing](#)
- [pytest getting started](#), [Playwright introduction](#), [GitHub Actions](#)
- [OpenAI structured outputs](#), [Anthropic structured outputs](#)

Later topics include production identity/authorization, managed secret delivery, retention and observability, broader AWS coverage and larger independently reviewed model evaluations. Kubernetes, LangGraph and vector databases are deferred, not hidden prerequisites.

Operations and troubleshooting

The supported deployment is a local workspace bound to loopback. Docker Compose runs PostgreSQL, migrations, API, worker and web. No AWS or model key is required for readiness or replay. Database credentials in `.env.example` are disposable local defaults, not cloud/API secrets.

Startup, ports and migrations

Install Docker with Compose v2 and Git, then work from the app root. On macOS/Linux:

```
test -f .env || cp .env.example .env
docker compose up -d --build
docker compose ps
```

On Windows PowerShell:

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
docker compose up -d --build
docker compose ps
```

The web UI is at <http://127.0.0.1:5173>, API at port 8000, PostgreSQL at 55432 and optional workload at 8080. The experiment runner owns temporary port 18080. Compose waits for PostgreSQL and runs alembic upgrade head before API/worker startup. Inspect docker compose logs --tail 100 api worker for actual stages/errors. docker compose --profile workload down stops the app while preserving named volumes.

For native development, the README bootstraps Python 3.12 and uv 0.12.23, then uv sync --frozen --group dev --group docs. Start only docker compose up -d db, run .venv/bin/alembic upgrade head, and launch .venv/bin/uvicorn proofops.api.app:app --host 127.0.0.1 --port 8000, .venv/bin/proofops-worker and frontend npm run dev in separate terminals. PowerShell uses .venv\Scripts\ executables. Stop container API/worker/web before using the same native ports. [README](#) contains exact fresh-setup, test, replay and cleanup commands.

Environment variables

Variable	Meaning / default
DATABASE_URL	PostgreSQL connection; host development uses port 55432. Compose replaces the host with db.
ARTIFACT_DIR	Dedicated directory inside application artifacts/; rejects research, code, evaluator and trust directories.
AI_MODE	off by default; live enables only otherwise eligible configured calls.
OPENAI_API_KEY / ANTHROPIC_API_KEY	Empty secret environment variables. Never committed or logged.
ALLOWED_PROVIDERS	Comma-separated explicit provider allowlist; empty blocks dispatch.
CHEAP_PROVIDER / CHEAP_MODEL	Configurable first attempt; default provider OpenAI, exact model ID unconfigured.
STRONG_PROVIDER / STRONG_MODEL	Configurable bounded escalation; default provider Anthropic, exact model ID unconfigured.
MODEL_PRICES_PATH	Exact model/capability/USD price records inside config/; current file has no live entries.
RATE_CARD_PATH	Optional CLI-only dated rate-card override inside the app; empty preserves bundle rates. Explicit --rate-card takes precedence. API imports always use the bundle's own rates.
AI_BUDGET_USD / AI_MAX_TASK_COST_USD	Positive explicit overall and per-task ceilings required for live dispatch. Defaults zero.
MODEL_TIMEOUT_SECONDS / MODEL_MAX_OUTPUT_TOKENS	Bounded SDK deadline/output; defaults 20 seconds / 1200 tokens.
AWS_ACCOUNT_ID / AWS_REGION	Expected STS account and region; no account configured, region defaults ap-south-1.
AWS_PROFILE / AWS_CLUSTER / AWS_SERVICE / AWS_LOG_GROUP	Optional normal credential-chain profile and exact collection scope.

Variable	Meaning / default
AWS_LOOKBACK_SECONDS / AWS_MAX_CALLS	Default 3600-second lookback and 12 calls; code also caps pages, results and time.
CORS_ORIGINS	Only local web origins by default. Unknown origins are rejected.
JOB_LEASE_SECONDS / JOB_TIMEOUT_SECONDS	Default 120-second lease, 180-second total job deadline.
MAX_BUNDLE_BYTES / MAX_ARTIFACT_BYTES / MAX_BUNDLE_FILES	Default 5 MiB total, 1 MiB per JSON artifact, 24 ZIP entries.

Each input bundle carries its dated `rates.json`. The CLI can use an explicitly configured `RATE_CARD_PATH` or `--rate-card` override and records its hash; this is not a live AWS pricing discovery feature. The shipped environment leaves the override empty so evidence bundles retain their own rate basis.

Diagnosis

Symptom	Inspect / resolve
Backend is unavailable	<code>docker compose ps</code> and <code>docker compose logs --tail 100 api</code> ; verify port 8000 is free.
Readiness returns 503	Start PostgreSQL and run <code>alembic upgrade head</code> ; missing model keys do not affect readiness.
Review remains queued	Start the worker and inspect <code>docker compose logs --tail 100 worker</code> . A queued job has no fabricated report.
A worker died	Restart it. Expired leases are reclaimed up to three claims; dispatched charges stay uncertain.
PostgreSQL connection fails in a sandbox	Permit the specific local database test/command. Do not change to SQLite to bypass transaction checks.
Guard validation unavailable	Install pinned <code>ConfTest 0.71.0</code> or rebuild the API image. Both <code>0.71.0</code> and the official Docker <code>v0.71.0</code> display format are accepted.
Review collects evidence	Inspect source status, sample count, freshness, exact <code>commit/image/config/profile/dependency</code> hashes and populations.
Synthetic replay requested as live	Supply genuinely applicable recorded AWS observations and a separately approved live contract; fixture approvals do not become real approvals.
AI unavailable	Inspect the advanced trace: allowed provider/model, current price entry, keys, explicit budget, refusal/truncation or quota. Deterministic findings remain available.
Budget stays reserved	Reconcile the provider's actual usage for the exact attempt. Never refund an ambiguous timeout/crash merely to retry.
An old export no longer applies	Replay reproduces history. Request a new review for changed commits, policies or expired evidence.
Workload runner refuses an existing directory	It protects recorded evidence. Use a new series for a fresh experiment, or resume compare with matching frozen inputs.

The budget ledger uses a fixed configured budget identity so restarting never resets spend. Lowering the environment limit is honored; increasing an existing ledger's limit requires a separately recorded operator/database maintenance decision. No unauthenticated public budget-reset endpoint exists. Reported usage may exceed a reservation if provider billing differs; this is recorded and blocks subsequent planned spending as appropriate.

Retention and recovery

Local data remains until the operator removes it. There is no production retention scheduler. Save needed sanitized review ZIPs before removing this app's volumes. `docker compose down` preserves volumes; `docker compose down --volumes` intentionally erases this app's database/artifact volumes. Research inputs are outside these volumes and must remain untouched.

API errors omit raw provider bodies and secret-bearing input values. Database operations use connection, pool, statement and lock timeouts. SDK automatic retries are disabled; the router accounts for at most two attempts. Model uncertainty never changes the deterministic outcome.

Before team hosting

This local application has no signup or user/tenant authorization. A team deployment would need authentication, role-specific approvals, account isolation, CSRF/session controls where applicable, secure secret delivery, encrypted storage, retention, observable operations and a reviewed deployment workflow. These are future deployment requirements, not implemented enterprise controls.

Owner and interview walkthrough

Allow about ten minutes. Use a running local app at <http://127.0.0.1:5173>; no cloud or model key is needed. This is a demonstration of implemented behavior, not recorded peer usability feedback.

1. State the problem: an engineer needs to connect a proposed ECS cost change to dated evidence and an approved operating contract. The system is advisory and reviews one explicitly mapped service.
2. Run the **valid-resize** replay. Show its synthetic origin, recorded reference time, 2 vCPU / 4 GiB baseline and 1 vCPU / 2 GiB candidate. The synthetic rate card produces an estimated 50% task CPU/memory reduction with unchanged task-hours. Point to excluded charges and passing fixture workload checks.
3. Open the **unsafe-resize** replay. The 1,024 MiB candidate breaches an approved-in-fixture 2,048 MiB floor. `revise_change` wins even though the arithmetic suggests a larger reduction. Neither AI nor operator enthusiasm can override the finding.
4. Open **incomplete-evidence**. A 20-request artifact cannot satisfy a 10,000-correct-request requirement; missing/stale/incompatible sources remain visible. Explain `collect_evidence` and the exact next evidence needed.
5. Prepare a guard draft from the applicable report, inspect its scope and bound, run fixtures and export. Show that the app has no activation button. Run `proofops guards test` to reproduce the deterministic guard; explain separate protected source review.
6. Enter a different candidate commit in the applicability check. The old approval becomes historical. Export and replay the original bundle to show that reproducibility and current applicability are separate questions.

7. Record an operator disposition, then load the FOCUS sample twice. Show one import, negative costs and exact totals. Explain that this public multi-provider sample is not the demo service's bill and not realized savings.

8. Show `docs/results/` and the local workload manifests. Distinguish actual local OOM/recovery and three comparison pairs from synthetic AWS review evidence. Present provider-policy cells as unrun; do not invent cheap/strong quality or pricing.

Questions the owner should be able to answer

- **Why not ask an LLM to approve the resize?** Arithmetic, scope and known policy violations have deterministic oracles. Prose can be plausible but causally wrong; explanation validation is not an approval authority.
- **Why PostgreSQL instead of only files?** Concurrent leases, idempotency and budget reservations need transactional state; files remain useful for portable immutable exports.
- **Why only one guard family?** A typed fixed template makes scope, exceptions and fixture coverage inspectable. Arbitrary generated Rego would broaden both trust and verification requirements.
- **Why not average p95?** A percentile summarizes a distribution. Averaging per-run percentiles is not the percentile of combined requests and can hide a bad run or request class.
- **What happens after a timeout?** Unknown provider charges remain pending; the worker does not assume zero cost and retry indefinitely.
- **What does a hash prove?** Exact byte identity. It does not prove that uploaded measurements are truthful or that an old observation applies now.
- **What would change for a team?** Add authenticated roles, approval separation, protected CI/workflows, secure secret storage, retention and account isolation. None is implied by loopback binding.
- **What was actually measured?** Local test/adaptor contracts, deterministic synthetic evaluation and local Docker workload behavior. Live AWS/provider quality, remote CI and uncoached human use are still unrun.

Glossary

Contract: versioned requirements and applicability. **Coverage:** whether supported evidence can answer the question. **Origin:** how evidence was obtained. **Replay:** deterministic reproduction at a recorded time. **Task-hours:** billable running-task time. **Guard:** scoped deterministic constraint. **Lease:** expiring worker ownership. **Reservation:** money set aside before dispatch. **Uncertain charge:** possibly billed attempt awaiting reconciliation. **Oracle:** independently justified expected result. **p95:** latency at or below which 95% of the sampled requests fall. **Abstention:** an explicit request for evidence or a coverage result. **Semantic correctness:** whether the meaning follows from evidence, beyond JSON validity.

Data sources and boundaries

- **FOCUS sample:** FOCUS-Sample-Data contributors, [original repository](#), revision `adbdd17a132984d6e8583c149c236d2199c3f5bc`, `FOCUS-1.0/focus_sample.csv`, retrieved in the supplied research on 2026-10-05. Unmodified copy in `fixtures/billing/focus_sample.csv`. [CC BY 4.0](#), local license in [licenses/FOCUS-license.md](#). No endorsement implied. A 1,000-row multi-provider USD sample, not the demo service's bill.
 - **Development knowledge cards, eight examples and answer schema:** original supplied ProofOps research. Source links are retained per card. Examples remain development material and never serve as held-out evaluation results.
 - **Three shipped review replays:** original synthetic application fixtures generated by `scripts/generate_fixtures.py`. Prices, incidents, approvals and workload measurements are illustrative. No AWS run produced them.
 - **RCAEval:** the research contains an MIT-licensed index and three controlled benchmark episodes. No RCAEval labels, index, fault-bearing paths or evaluator-only files are copied into the runtime. Optional diagnosis is deferred.
 - **ITBench/Infracost research:** scenario/parser design references only; no upstream environment or Terraform has been executed. Infracost import is optional and deferred because the versioned rate-card path is implemented.
- Research checksums are saved in `docs/research_checksums.json`; `python scripts/check_research.py` verifies the original files remain unchanged.
- **Workload script:** `testing/virtual_users.js` adapts the original supplied ProofOps research script. The app adds observed run accounting and explicit per-class results. Research originals remain unchanged; all local experiment outputs are labelled `local_observation`.
 - **PDF typography:** embedded Bitstream Vera fonts are supplied by the locked ReportLab distribution. The redistribution notice is preserved in [licenses/bitstream-vera-license.txt](#). Documentation diagrams are original vector drawings generated locally; no third-party screenshots or generated raster art are used.
 - **UI icons:** Lucide React, locked in `frontend/package-lock.json`, supplies the interface icon components under its ISC license. Dependency license notices remain in the installed package; [Lucide license](#).